软件整体架构与界面设计报告

项目名称：活动票务与报名系统

学号：241220104　姓名：张士博　日期：2026 年 11 月 13 日

课程名称：软件工程　任课教师：王豫　专业：计算机科学与技术

本实验在实验一确认的需求（requirements.md，35 条编号需求）基础上进行软件整体架构设计与界面设计，UML 图全部用 PlantUML 绘制，.puml 源文件与 .png 图片同名成对提交。

1　设计概述

本项目是实验一选定的《活动票务与报名系统》：面向校内讲座、演出、志愿服务培训这类活动，把活动发布与审核、多条件查询、三步报名、名额与候补递补、现场检票、名单导出与容量统计放在一个系统里，使用者分参与者、组织者、检票人员、管理员四类角色。业务范围、字段口径与需求编号全部沿用实验一经甲方确认的需求文档，本设计不新增业务需求，只为已确认的需求给出结构、行为、交互与界面定义。

1.1　设计输入与依据

设计输入是实验一经甲方确认的需求整理文档 requirements.md。该文档共有 35 条编号需求，其中已确认 27 条、已变更 3 条、待确认 2 条、已拒绝 3 条（REQ-014、REQ-016、REQ-017 作废，编号保留不再复用）。本次设计只对已确认与已变更的 30 条需求给出结构、行为和交互定义；REQ-034（缺席次数与报名资格限制）与 REQ-035（变更与递补通知渠道）仍为待确认，只保留接口位置，不做实现承诺。

实验一确定的四条硬性口径在本设计中直接落成结构约束，不再重新讨论：报名与名额的守恒式（总名额＝剩余＋已确认＋待支付＋递补待确认，REQ-027）、名额只能在一个入口内变动（REQ-019）、递补确认时限 30 分钟与两次逾期规则（REQ-021）、时间判定一律以服务器时间为准（REQ-010）。

1.2　设计原则

（1）名额只有一个入口。任何改变 remainingCount 的动作都必须经过 CapacityService 与它持有的事务，界面、导入、定时任务概莫能外，从结构上排除超卖与漏减（REQ-019、REQ-027）。

（2）状态只能由状态机决定。Enrollment 的 status 字段不提供公开写入口，转换由 EnrollmentStateMachine 判定，禁止的转换在服务端拒绝，避免各处代码各自改状态（REQ-024）。

（3）校验放在服务端。角色、归属关系、限购张数、票种停售、活动状态、页面版本号的校验都在服务端完成，界面只做提示，防止绕过页面直接调接口（REQ-002、REQ-023、REQ-028）。

（4）关键操作与日志同事务。十三类关键操作的日志写入与业务写入在同一事务内完成，日志写失败则业务回滚，不允许出现业务成功但无日志的情形（REQ-015、REQ-032）。

（5）时间只认服务器。报名起止、30 分钟递补时限、导出链接 30 分钟时效、日志时间段查询，全部取 ServerClock，客户端时间不参与判定（REQ-010、REQ-021、REQ-029）。

（6）为可测试而设计。每条需求都能落到某个模块、某张表、某个界面或某条转换上，第 6 章的追踪矩阵用于逐项核对。

1.3　软件主要功能

系统面向四类角色提供功能，另有无人触发的系统内部动作。需求编号集中在第三列，便于与实验一对照。

表 1　按角色归并的主要功能

角色	主要功能	对应需求

参与者	手机号注册与登录；多条件组合查询活动与剩余名额；三步完成报名并保存提交时单价；截止前取消本人的已确认或待支付记录；名额已满时登记候补并查看位次；在 30 分钟内确认递补；查看站内通知与本人相关操作记录	REQ-001、006、007、008、009、015、021、033、035
组织者	提交资质材料待审；创建活动草稿并设置场次、票档、限购与开售时间；提交活动审核；开售后按下限调整名额、单票种停售；申请延长报名截止；修改关键信息后重新送审；取消整场活动并生成应退名单；标记线下收款与退款；导出名单快照与查看容量统计；按活动授权检票人员	REQ-003、004、005、010、012、013、022、025、026、027、031
检票人员	凭本活动授权，按报名编号、姓名后四位或入场码核验；确认入场并记录时间与经办人；对误签错签发起撤销申请；查看本活动签到数与签到率	REQ-011、013、025
管理员	审核组织者资质与活动发布；审批延长报名与关键信息变更；审批并执行撤销签到；执行活动取消与账号解锁；按四条件组合查询并导出日志；结束后标记缺席；运行名额守恒对账	REQ-004、010、011、012、015、026、028、030、032
系统内部	名额原子扣减与并发串行化；重复提交幂等返回；过期页面版本校验；候补队列自动递补与逾期顺延；站内通知生成；名单快照与下载时效；口令加盐哈希与失败锁定；日志只追加并保留一年	REQ-013、015、019、020、021、023、027、028、029、032

实验二的任务书要求模块划分必须覆盖实验一中的基本要求模块。把实验一经甲方确认的 30 条有效需求按要做的事情归类，是六类：账号与权限、活动发布与审核、场次与票种、报名与候补递补、签到与名单、查询与统计。本设计为这六类各设了主责模块，依次是 M1、M2 与 M8、M2、M4 与 M5、M6 与 M7、M3 与 M7，一类不删；M9 通知、M10 日志、M11 定时任务、M12 公共契约四个支撑模块承接其中的横切事项。每条需求落在哪个模块、哪张表、哪个界面，第 6 章的追踪矩阵逐项给出，划分的理由见 2.2 节。

2　模块划分与架构说明

2.1　模块划分

系统按“界面—接口—业务服务—领域—基础设施—公共契约”六层划分，业务服务层切成 M1 至 M10 十个模块，另设 M11 定时任务与 M12 公共契约两个支撑模块。划分结果与职责、对应需求、主责界面和数据表如下。

表 2　模块划分与需求、界面、数据表的对应

编号	模块	职责边界	对应需求	主责界面	主要数据表
M1	身份与权限	注册、登录、失败锁定、角色判定、检票授权、越权拒绝与解锁	REQ-001、002、025、028	界面 1、界面 6、界面 8	account、checkin_grant
M2	活动管理	活动与场次票种的增删改、草稿、提交审核、停售、名额调整、活动取消	REQ-003、004、005、012、022、026	界面 6、界面 7	activity、activity_session、ticket_type
M3	查询与展示	多条件组合查询、状态过滤、剩余名额展示、查询响应指标	REQ-006、018、033	界面 2、界面 3	activity、ticket_type（读）
M4	报名处理	报名表单令牌、幂等标识、版本校验、生成记录、取消报名	REQ-007、008、020、023	界面 3、界面 4、界面 5	enrollment（含 request_token、page_version 列）
M5	名额与候补	名额原子扣减与释放、候补队列、自动递补、守恒校验与对账	REQ-009、019、021、027	界面 4、界面 5、界面 9	ticket_type、waitlist_position、enrollment
M6	签到核验	编号与入场码核验、重复签到判定、撤销申请与审批执行	REQ-011、024	界面 8、界面 7	enrollment、checkin_grant、revoke_request
M7	名单与统计	容量统计、名单快照导出、下载权限与时效、应退名单	REQ-013、027、029、031	界面 9、界面 6	export_snapshot、refund_list
M8	审核工作流	资质审核、活动审核、延长报名审批、关键信息变更重审	REQ-004、010、025、026	界面 7	qualification_review、extension_request、key_info_change
M9	通知	站内通知生成与读取，外部渠道仅预留接口	REQ-010、012、021、026、035	界面 5、界面 7	notice
M10	日志与审计	十三类关键操作记录、四条件组合查询、只追加、保留一年	REQ-015、028	界面 7	audit_log
M11	定时任务	报名截止控制、递补计时、缺席标记与名单关闭	REQ-010、021、030	无界面（后台）	enrollment、activity
M12	公共契约	枚举、常量、错误码与拒绝原因目录，供各层共用	REQ-033	全部界面的提示文案	—

两点说明。其一，M12 公共契约没有业务逻辑，但它把界面提示文案集中到一处，实验一 REQ-033 要求八类拒绝场景都要给出原因和下一步，散在各页面里写会不一致，因此抽成目录。其二，M11 定时任务单独列出，是因为 REQ-010 的截止判定、REQ-021 的 30 分钟时限、REQ-030 的缺席标记都不能依赖用户是否在线；把这三类“由时间而非人触发”的逻辑集中到一个模块，便于按服务器时间统一测试。

2.2　这样划分的理由

（1）把名额与候补（M5）从报名处理（M4）里分出来。报名是一次用户动作，名额是全局资源：同一票种上可能同时有并发抢名额、候补递补、组织者调名额、活动取消批量释放四种变更。把资源变更收在一个模块，才能把 REQ-019 的原子性和 REQ-027 的守恒式写成一处可核对的规则，而不是散在四个流程里各自实现。

（2）签到核验（M6）与撤销分开管理。核验由检票人员执行，撤销必须“检票人员申请＋管理员审批”，两者权限不同、留痕不同，故撤销申请作为独立对象（revoke_request）保存，审批通过才改状态，避免把“申请”当成“执行”。

（3）审核工作流（M8）集中四类审核。资质审核、活动审核、延长审批、关键信息变更重审的共同点是“申请—审批—生效”三段式，共用一套状态和日志口径；分开写会出现四套相似的通过/驳回代码，实验一里已经因为分散表述造成过歧义。

（4）通知（M9）与业务解耦但同库保存。业务动作只写 notice 表，不直接调外部渠道；REQ-035 的外部渠道待甲方确认，将来接入时只替换发送端，业务代码不动。

（5）日志（M10）独立成模块且只追加。REQ-015 要求十三类操作留痕、不提供修改删除入口，接口上只给 append 与 query 两个方法，物理上单独一张表并按时间段建索引，配合一年后归档，避免与业务表混在一起被误改。

（6）界面按角色拆成四套而不是做一套通用界面。四类角色的动作集合差异大（参与者三步报名、组织者批量管理、检票人员单屏核验、管理员审核队列），共用界面会带来大量“看不见但存在”的入口，越权风险高。拆分后界面上的每个入口都能在权限表里对应到角色（REQ-002）。

2.3 模块之间的逻辑关系

整体是分层单体＋后台定时任务的结构：请求自上而下经过界面层、接口层、业务服务层落到领域层，基础设施层被领域层和服务层调用，反向依赖禁止（图见 UML 包图.png）。三条主链如下。

表 3 三条主链的模块交互

主链	调用顺序	关键约束与理由
报名	界面 3/4 → API_Enroll → EnrollmentService → CapacityService（同事务）→ TicketType 行锁 → AuditLogService →（不足时）WaitlistQueue	名额扣减与记录写入必须在一个事务里（REQ-019）；幂等标识在 EnrollmentService 判定，不放界面（REQ-020）；版本号校验也在服务端（REQ-023）
递补	EnrollmentService.cancel 或 M11 定时任务 → CapacityService.releaseAndPromote → WaitlistQueue.head → Enrollment 状态转换 → NotificationService →（30 分钟）WaitlistPromotionJob 回查	30 分钟时限由服务器时间与通知发出时间之差判定，客户端不计时（REQ-010、021）；逾期顺延与两次逾期规则放在同一个 Job 里，避免两处实现不一致
导出	界面 9 → API_Check → ExportService →（读同一时点快照）票种与报名记录 → FileStorage → AuditLogService	快照与统计必须同一次读取（REQ-013、027）；下载链接 30 分钟时效与权限校验在服务端（REQ-029）；越权导出拒绝并记日志（REQ-015、028）

模块之间只允许两种协作方式：同层服务之间显式调用（如 EnrollmentService 调 CapacityService），以及领域对象通过状态机与规则类自我校验（Enrollment 调 EnrollmentStateMachine、TicketType 调 CapacityRule）。不允许界面直接访问仓储、也不允许领域层反向调用界面组件，包图中箭头方向即依赖方向。

3 软件架构的 UML 图设计

本章共给出八张图，按 UML 的三种视角归类：结构性视图四张（包图、组件图、类图、服务类图），行为性视图三张（用例图、活动图、状态图），交互性视图一张（时序图）。类图为必画内容。所有图用 PlantUML（1.2026.10，本地 OpenJDK 17 渲染）绘制，源文件与图片同名成对放在 UML 目录，每张图的名称标在图注中。

表 4 图清单与视角分类

图名	视角分类	文件	要说清的问题
类图	结构性	UML 类图.puml / UML 类图.png	领域实体的属性、方法与关联，名额与状态的归属
服务类图	结构性	UML 服务类图.puml / UML 服务类图.png	六个服务接口与实现类、定时任务的位置
包图	结构性	UML 包图.puml / UML 包图.png	六层划分与依赖方向
组件图	结构性	UML 组件图.puml / UML 组件图.png	模块之间通过哪些动作互相调用
用例图	行为性	UML 用例图.puml / UML	四类角色与定时任务能做

		用例图.png	什么
活动图	行为性	UML 活动图.puml / UML 活动图.png	报名到候补的分支与校验顺序
状态图	行为性	UML 状态图.puml / UML 状态图.png	报名记录九个状态与允许、禁止的转换
时序图	交互性	UML 时序图.puml / UML 时序图.png	并发抢名额、幂等、释放与递补的消息顺序

3.1 类图（必画）

类图给出领域层的实体、规则类以及它们之间的关系。两个状态枚举不画成图上的独立节点——枚举有九个取值，画进类图会把画布撑大、字压小，取值改列在表 5 里逐项说明。Account 是四类角色的父类，登录失败次数与锁定时间放在父类，四种角色共享同一套口令规则（REQ-028）；Organizer 只多一个资质状态字段，因为资质审核结论由 M8 的记录表承载，不冗余在账号上。Activity、ActivitySession、TicketType 是实验一确定的三级结构（REQ-005），名额与限购张数保存在票种上而不是活动上，一个活动可以有多个场次、每个场次多个票档，并发扣减的对象也正是票种这一行。

Enrollment 是本设计的中心：它同时挂参与者与票种，保存提交时的单价快照（paidAmount）、一次性请求标识（requestToken）和页面版本号（pageVersion）——后两个字段是 REQ-020、REQ-023 落地成数据的结果，没有它们就无法在提交时判定“重复提交”和“过期页面”。WaitlistQueue 与 CapacityRule 不是数据表，而是票种上的两个行为入口：队列顺序与守恒校验只此一处。

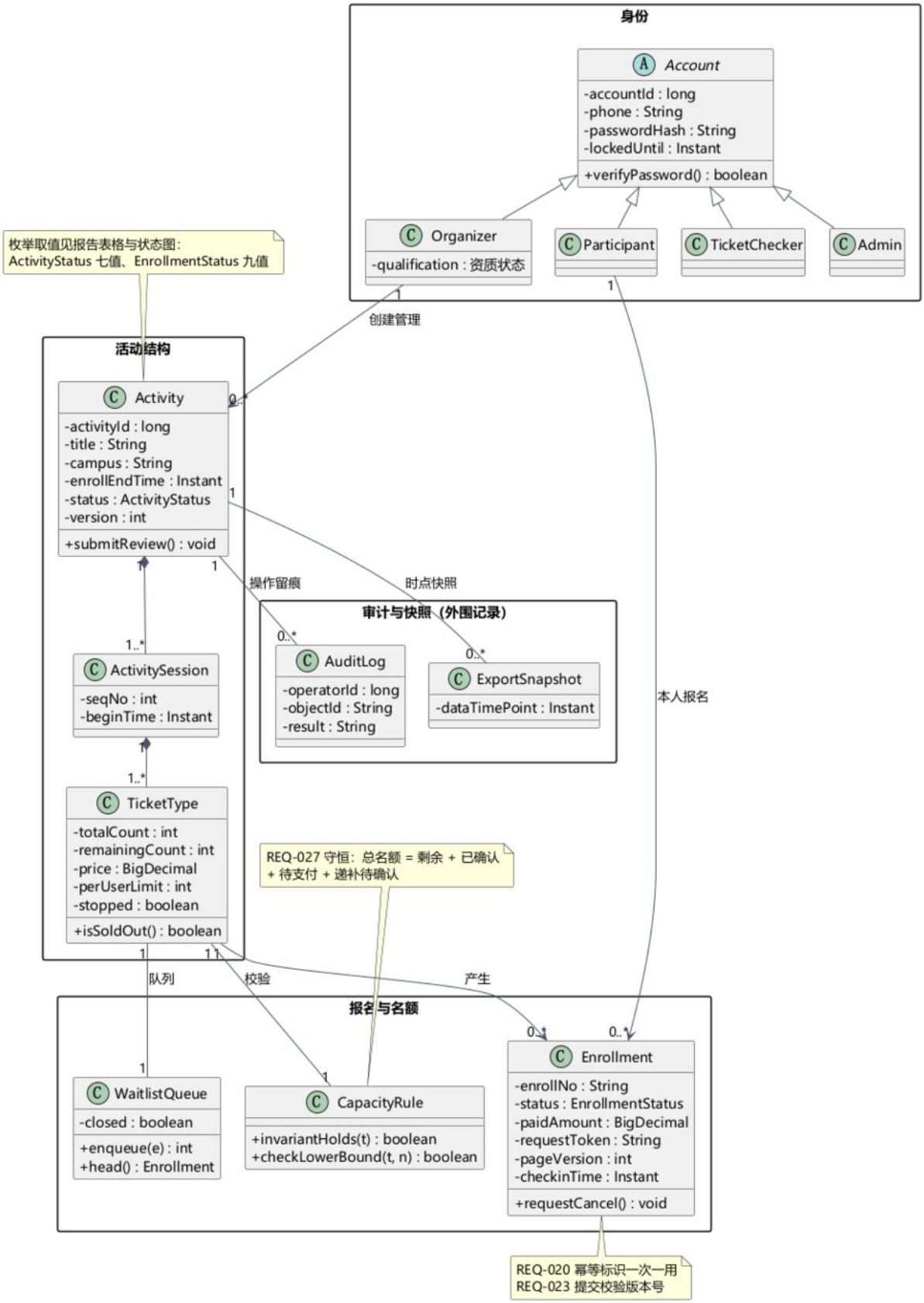


图 1　类图（UML 类图.png）：领域实体与规则类

两个枚举的取值在下表列出，状态图与追踪矩阵均按此口径。

表 5　枚举取值与口径

枚举	取值	说明
ActivityStatus	草稿、待审核、已发布、报名中、已截止、已结束、已取消	与实验一图 2 的活动状态链一致；“已满”不是持久状态，由票种剩余名额推导，用于查询条件的展示（REQ-006）
EnrollmentStatus	待支付、已确认、候补中、递补待确认、已签到、已缺席、已取消、已退款、候补已结束	前八个即 REQ-024 列举的状态；第九个“候补已结束”是本设计补充，理由见 3.7 节
资质状态	待提交、待审核、已通过、已驳回	待审核与已驳回的组织者不能创建活动（REQ-025）
审核结论	通过、驳回（须填理由）	用于活动审核、延长审批、资质审核与关键信息变更重审四处（REQ-004、010、025、026）

设计取舍：Enrollment 与票种之间用普通关联而非组合，因为取消报名、活动取消批量终止都要保留记录本身；若做成 TicketType 的组成部分，删除票种会波及历史记录。ExportSnapshot 与 Activity 关联而不与 Enrollment 关联，因为快照的口径是“某活动时点的一组记录”，一次导出对应一条快照记录，文件里再按票种分列（REQ-013）。

3.2　服务类图

服务类图把 2.1 节的模块落到可调用的接口上。六个接口是界面能触达的全部入口，实现类的私有成员是它所依赖的仓储与时钟，WaitlistPromotionJob 没有对外接口，只被定时触发。这样划分的理由有两点：第一，接口数量少而方法名即操作名，权限校验可以按接口方法逐条配置，越权判断不必读实现代码；第二，CapacityService 与 AuditLogService 都被 EnrollmentService 直接引用，让“扣名额”和“写日志”在同一个实现类里被看见，避免有人绕过它去改表。

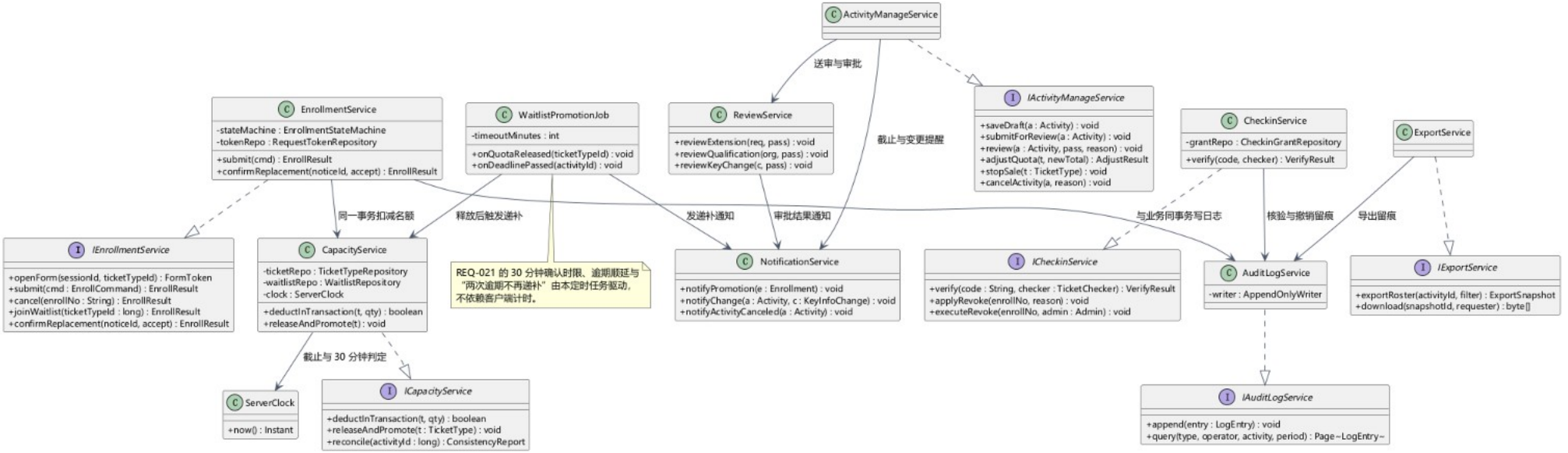


图 2　服务类图（UML 服务类图.png）：接口、实现类与定时任务

3.3　包图：分层与依赖方向

包图给出六个顶层包与依赖方向。界面层只允许调用接口层，接口层只做鉴权、参数校验与错误码翻译，业务规则一律下沉到服务层与领域层；基础设施层的五个组件（仓储、事务与行锁、文件存储、服务器时钟、口令哈希）被上层持有接口调用，自身不认识业务。公共契约包被各层引用，不含逻辑。这样分层的好处与本项目的风险点直接对应：并发、越权、时间判定三类问题各自有一个固定去处，后续编码时不必在每个功能里重复实现一遍。

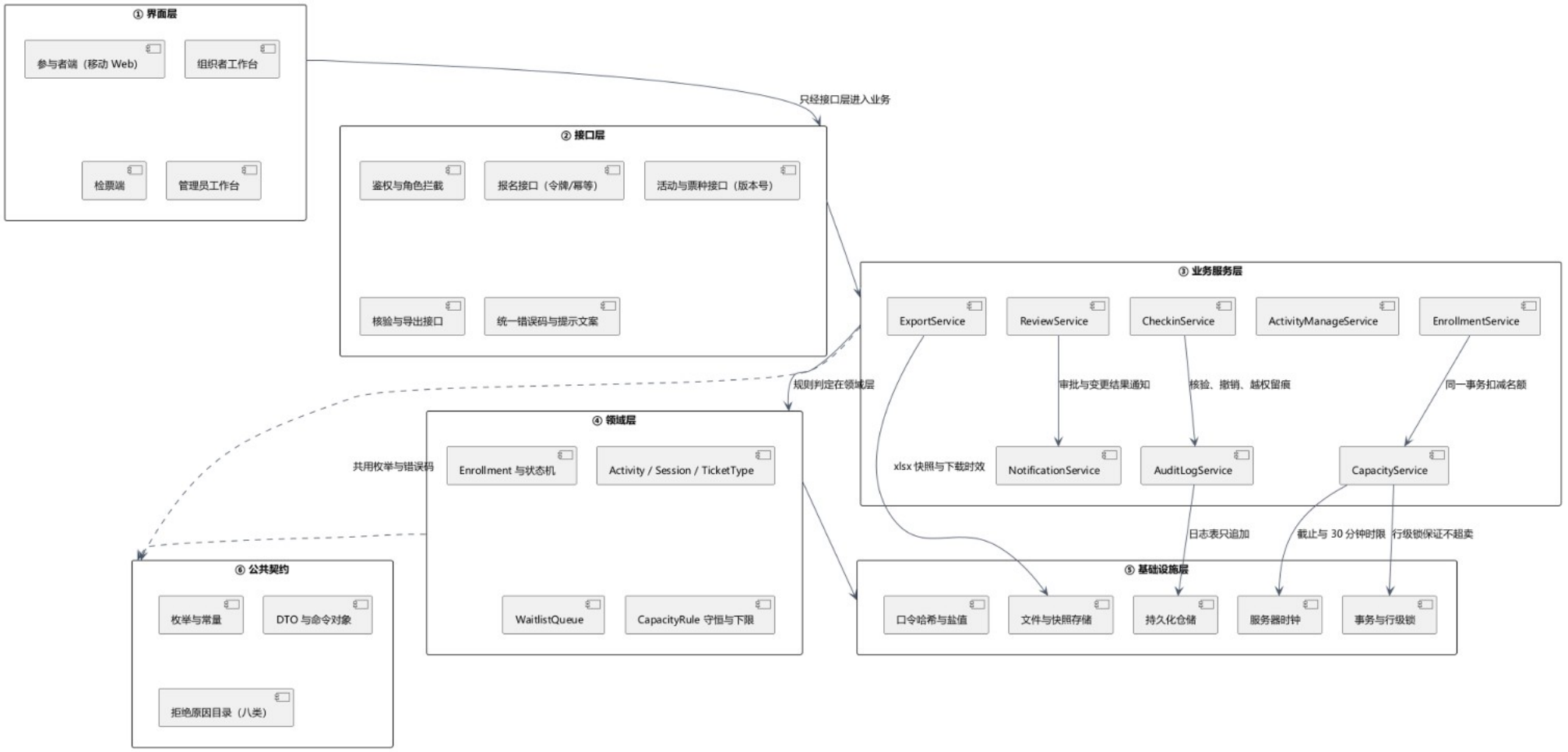


图 3　包图（UML 包图.png）：六层结构与依赖方向

3.4　组件图：模块之间的交互

组件图上的每条箭头都是一次实际调用，标注文字即调用内容。最需要说明的是三条：报名处理指向名额与候补的调用发生在同一事务内（REQ-019），因此这两个组件不能拆成两次网络请求；签到核验指向身份与权限的调用是每次核验都要做的（REQ-025 撤销授权后立即失效）；活动管理把审核请求交给审核工作流，审核结果只流向通知，生效写回仍由活动管理完成，这一去一回正是 REQ-010、026“审批通过才切换新信息”的顺序保证。日志组件被报名处理、签到核验、导出与统

计、审核工作流四个业务组件调用且只追加，不提供修改入口（REQ-015）；报名、名额调整等其余模块写日志的动作在包图上是同一条依赖（服务层指向基础设施层），若在这里全部画出会有十几条箭头交叉，反而看不出主链。

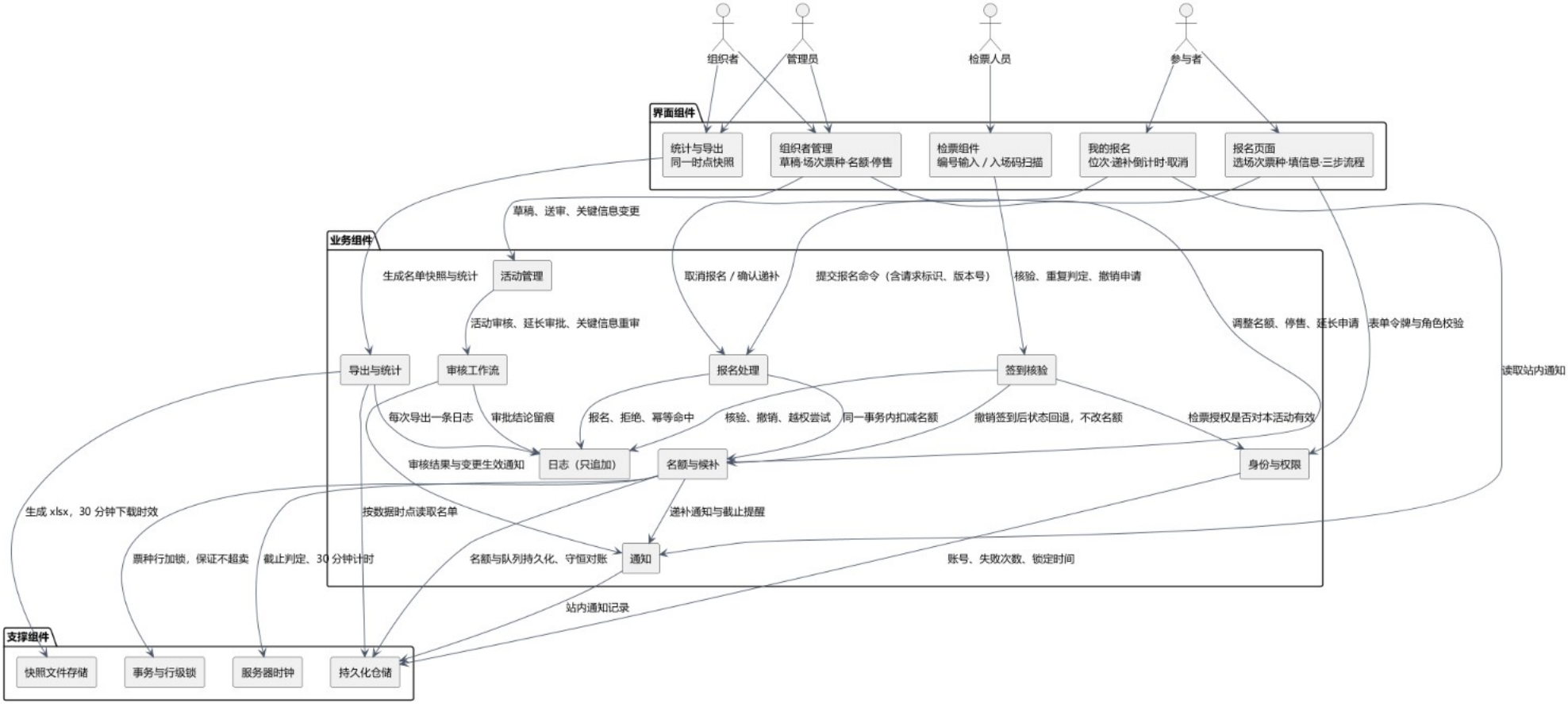


图 4　组件图（UML 组件图.png）：模块交互与接口

3.5 用例图

用例图按四类角色列出 21 个用例，另有三个 include 用例（权限校验、服务器时间判定、日志记录）与两个 extend 用例（名额已满转候补、重复提交返回原结果）。include 与 extend 的用意是把“每个用例都要做但容易被漏写”的公共动作显式画出来：实验一的 REQ-028 要求越权请求一律拒绝并记日志，若只写在权限模块里，验收时很难逐条对上，画进用例图后每个涉及写的用例都必须连到这两个公共用例。

定时任务作为独立参与者出现，连接“确认递补（超时处理）”与“标记缺席并关闭名单”两个用例，提醒这两个动作没有真人触发（REQ-021、REQ-030）。“后续”关系（创建活动→提交审核→审核发布、申请撤销→审批撤销、延长申请→审批延长）表达的是业务顺序而非包含关系，与 REQ-004、010、011 一致。



图 5　用例图（UML 用例图.png）：四类角色与定时任务

3.6　活动图

活动图描述一次报名提交从校验到落库的分支。三条设计原因：第一，格式、请求标识、版本与状态三重校验放在一个节点里顺序执行，顺序不可调换——先判幂等再判版本，重复提交才会稳定返回第一次的结果而不是“请刷新”（REQ-020 优先于 REQ-023）；第二，名额判断与记录写入在同一个加锁区段内完成，图上不出现“先扣名额再写记录”的两步，这正是 REQ-019 要求的原子性；第三，候补分支只在“名额不足且未截止未停售”时打开，截止后队列停止（REQ-021），日志与守恒校验排在每个成功分支的末尾，保证“业务成功必有日志”（REQ-015）。

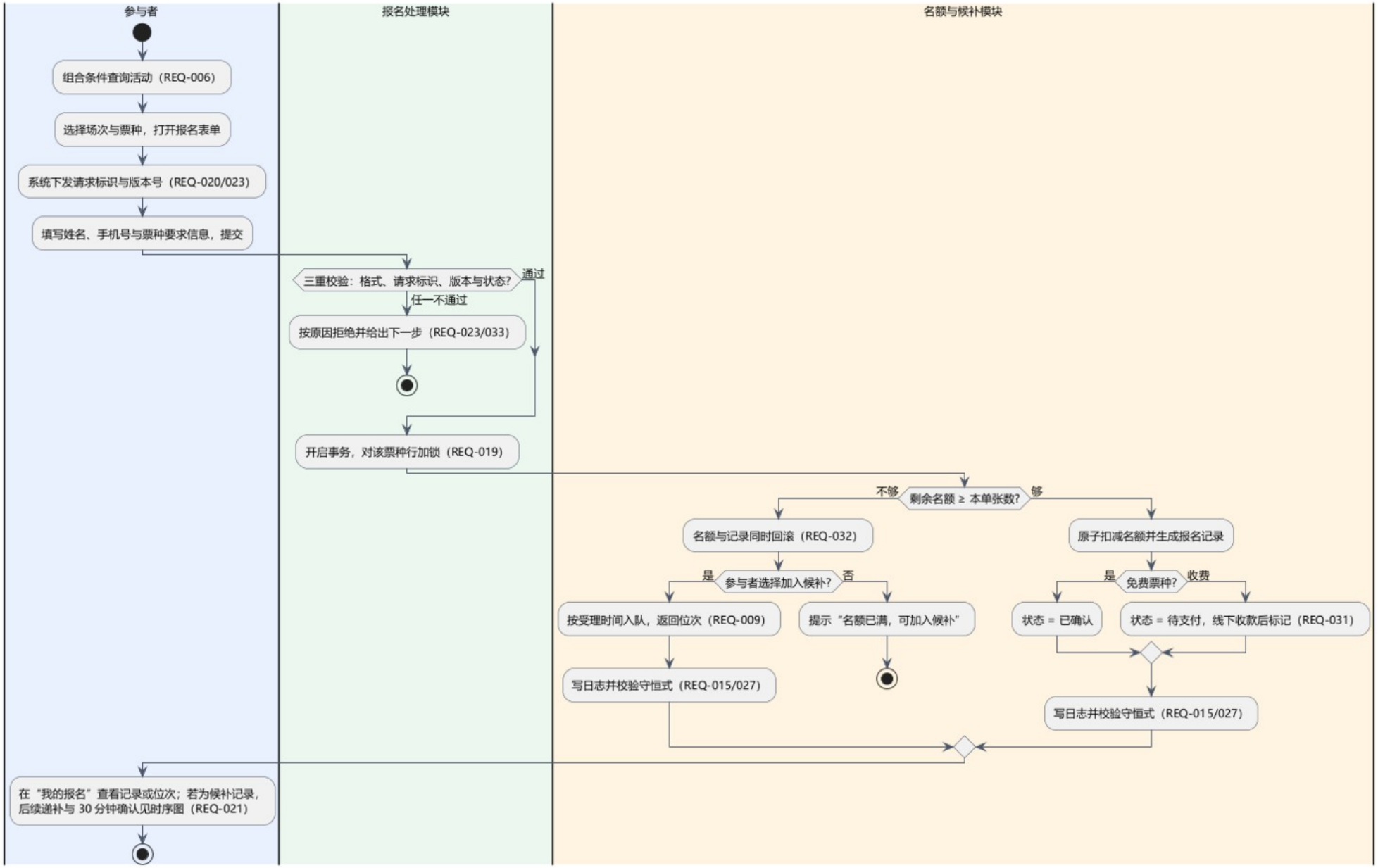


图 6　活动图（UML 活动图.png）：报名提交与候补分支

3.7　状态图

状态图给出报名记录的九个状态与全部允许转换，禁止转换以清单形式写在图内。这里有一个必须说明的设计决定：实验一 REQ-024 列举的状态是八个，但 REQ-021 又要求“报名截止后队列停止递补，剩余候补记录转为已结束”，即存在一个只属于候补记录的终止态。设计时先按八个状态画，发现“候补中”在截止后没有去向；若把它并入“已取消”，又与 REQ-008 的取消语义（参与者主动取消并释放名额）冲突，统计应退名单时会误纳。因此本设计把“候补已结束”作为第九个状态显式建出，只接受“候补中”转入，不再转出，也不参与名额守恒式（它不占用名额）。这一点建议在下一轮需求沟通时与甲方确认是否回写进 REQ-024 的清单，按实验一约定，需求编号一经使用不再改动，届时应新增编号而不是改写原条目。

表 6　允许的转换与触发者

转换	触发者	条件	对应需求
待支付 → 已确认	组织者	线下收款后标记	REQ-031
待支付 → 已取消	参与者或系统	本人取消；截止仍未支付	REQ-008、010
已确认 → 已签到	检票人员	核验通过且授权有效	REQ-011
已签到 → 已确认	管理员执行	检票人员申请且审批通过，必须留痕	REQ-011、015
已确认 → 已取消	参与者	截止前本人取消，释放名额触发递补	REQ-008、021
已确认 → 已缺席	定时任务	签到窗口关闭仍未核验	REQ-030
候补中 → 递补待确认	定时任务	队列首位且名额可用，锁定名额	REQ-021
递补待确认 → 已确认/待支付	候补者	通知发出后 30 分钟内确认	REQ-021
递补待确认 → 已取消	定时任务	30 分钟逾期，名额顺延	REQ-021
候补中 → 候补已结束	定时任务	报名截止，队列停止递补	REQ-021（本设计新增终止态）
任意有效状态 → 已取消	组织者或管理员	活动取消，已签到记录除外	REQ-012
已确认/待支付 → 已退款	组织者	线下退款并标记	REQ-031

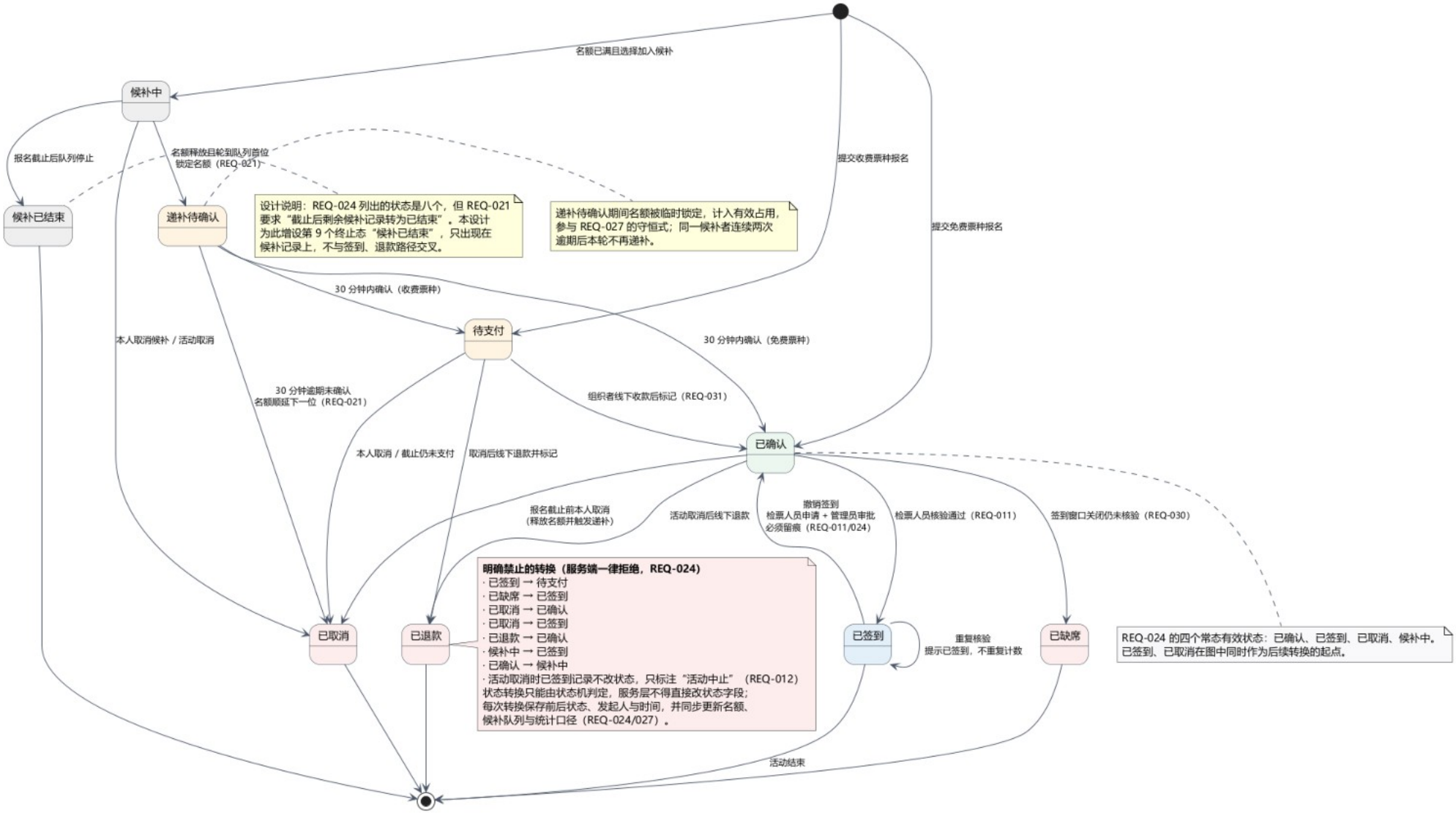


图 7　状态图（UML 状态图.png）：报名记录的九个状态

3.8　时序图

时序图取“抢最后一个名额”这一最脆弱的场景。甲的事务先获票种行锁并提交，乙的请求在锁外等待，获锁后读到剩余 0，于是被拒绝并给出候补入口——图上没有“先查后减”的两步，因为先查后减正是超卖的来源（REQ-019）。甲随后用同一标识重试，服务靠 request_token 的唯一约束命中，返回第一次的记录编号而不是新记录（REQ-020）。甲取消后，名额在同一事务里回到票种上，再由 CapacityService 取队列首位锁定并发通知；图下方的注写了两处收尾规则：30 分钟计时由后台任务按服务器时间回查、逾期顺延且同一人两次逾期本轮不再递补（REQ-021），以及日志写入失败时名额与状态一起回滚、请求标识保持可用让用户直接重试（REQ-015、032）。

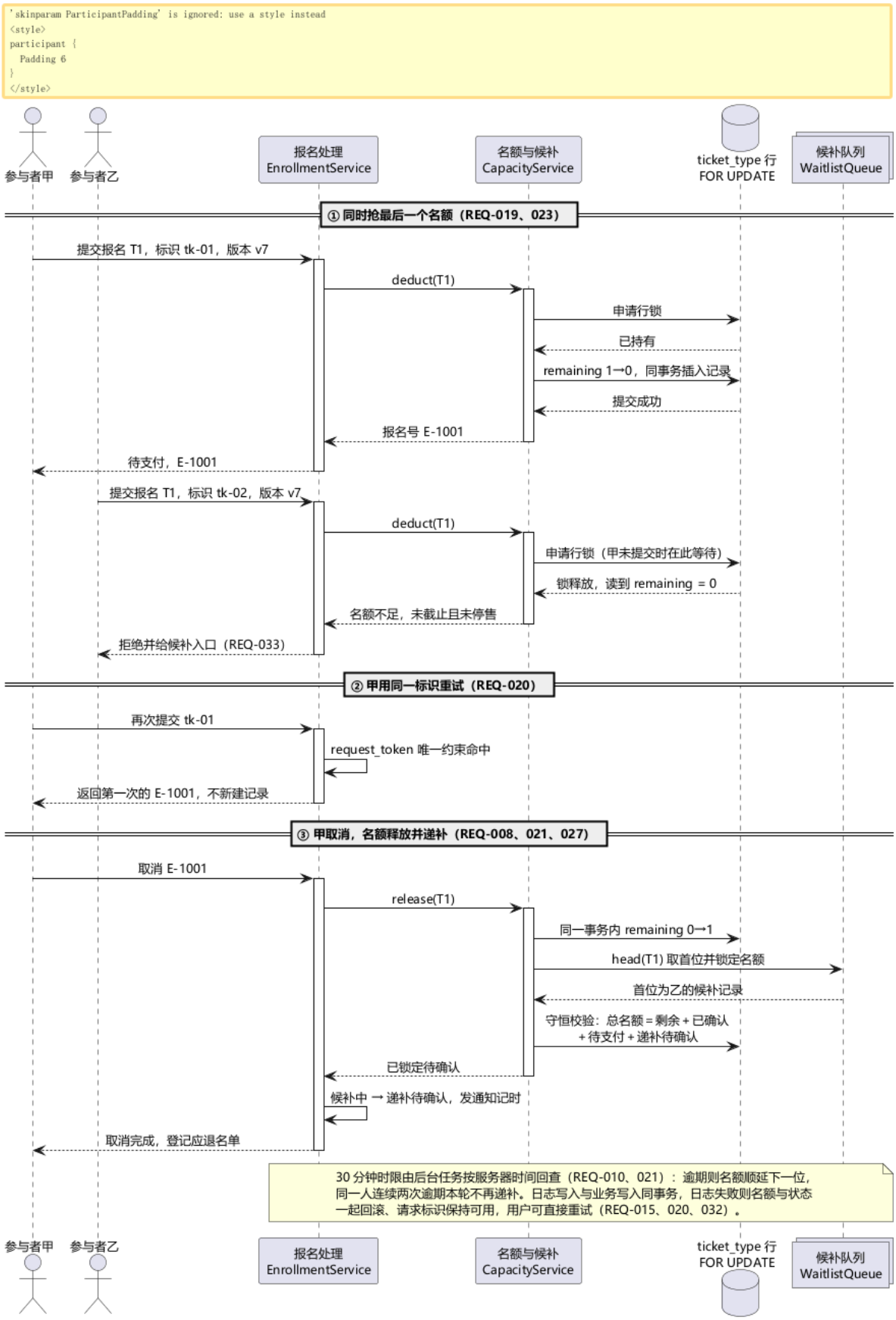


图 8　时序图（UML 时序图.png）：并发抢名额、幂等与递补的消息顺序

3.9　三类视图的覆盖核对

结构性视图四张、行为性视图三张、交互性视图一张，满足任务书“类图必画且三种类型各至少一种”的要求，报告内图数超过四张的下限。需要补充说明的一处不均衡是：交互性视图只有一张时序图，它选择覆盖并发抢名额场景，是因为该场景的消息交错最能暴露设计缺陷；签到核验与名单导出两条链的交互相对直白，用 2.3 节的调用表和组件图表达已够，后续编码时若发现导出与统计对账的时序有歧义，再补一张时序图。

任务书列出的十四种图里未采用的几种，各自的判断如下。对象图要的是某一时刻的实例快照，本项目的争议都在规则与状态上，快照帮不上忙；复合结构图刻画单个类内部的部件与端口，而 CapacityService 的内部结构用类图加表 7 的字段已经说清；部署图在本系统只有一种部署形态（REQ-018 给的单机配置加一个数据库），画出来是三个框两条线，信息量不如 2.1 节的六层表；剖面图用于定义建模扩展，不属于本项目的交付物；通信图与交互概览图表达的是同一批消息，时序图已经把锁等待与事务回滚的先后讲完，重复画一张只会增加两处对不上的风险。序列图与时序图是同一张图的两种译名，不另画。

4　数据的存储形式

任务书第五节要求设计尽可能保证基本功能的完备性、包括数据的存储形式，本节按此给出表级设计。存储选用关系库（MySQL 8 口径），理由是本系统的核心约束是行级一致性与事务：名额扣减要锁 ticket_type 的一行，幂等要唯一约束，日志要只追加，这三件事用关系库的约束与事务表达最直接。后续若改用其他存储，本节字段与约束仍可作为对照。

表 7　表清单与约束

表	用途	主键与唯一约束	关键索引	对应需求
account	四类账号与口令、失败次数、锁定时间	PK(account_id)；UK(phone)	idx_status	REQ-001、028
qualification_review	组织者资质审核记录	PK(id)；UK(org_id, submit_time)	idx_result	REQ-025
activity	活动主体与状态、报名起止时间、版本号	PK(activity_id)	idx_status_start、idx_campus_category；关键词检索原型用 LIKE 加前缀索引，量大后再换 ngram 全文索引	REQ-003、006、010、023
activity_session	场次	PK(session_id)；UK(activity_id, seq_no)	idx_begin_time	REQ-005
ticket_type	票种：总名额、剩余名额、单价、限购、停售、版本号	PK(ticket_type_id)；UK(session_id, name)	idx_activity；约束 remaining_count ≥ 0 AND ≤ total_count	REQ-005、007、019、022
enrollment	报名记录（含候补与递补记录）、单价快照、请求标识、页面版本、签到时间与经办人	PK(enroll_no)；UK(request_token)	idx_user_session_ticket、idx_status、idx_activity_checkin	REQ-007、008、020、023、024
waitlist_position	候补队列顺序（与 enrollment 一对一）	PK(enroll_no)；UK(ticket_type_id, position)	idx_queue(ticket_type_id, position)	REQ-009、021
revoke_request	撤销签到申请与审批结论	PK(id)	idx_status_activity	REQ-011
extension_request	延长报名申请与审批	PK(id)；UK(activity_id, new_deadline)	idx_status	REQ-010
key_info_change	关键信息变更前后值与重审结论	PK(id)	idx_activity_result	REQ-026
refund_list	应退名单与退款标记	PK(id)；UK(enroll_no)	idx_activity	REQ-008、012、031
export_snapshot	名单快照元数据（时点、导出人、行数、下载失效时间）	PK(snapshot_id)	idx_activity_time	REQ-013、029
notice	站内通知	PK(notice_id)	idx_user_read	REQ-035
audit_log	十三类关键操作日志，只追加	PK(log_id)	idx_type_operator_activity_time（四条件组合查询）	REQ-015、028

4.1 三处关键设计

（1）名额与记录的写入在同一事务。enrollment 插入与 ticket_type.remaining_count 更新在同一事务内完成，并对 ticket_type 行加锁（SELECT ... FOR UPDATE），使并发请求按受理顺序串行化；事务失败时两者一起回滚，不留“只扣名额”或“只写记录”的中间状态（REQ-019、032）。

（2）幂等靠唯一约束而不是判重查询。request_token 在 enrollment 上建唯一索引，重复提交第二次插入会直接触发约束冲突，服务捕获后返回第一次的记录；这样即使两个请求几乎同时到达也不会靠“先查后插”竞态生成两条记录（REQ-020）。

（3）手机号存明文、出口脱敏。参与者本人查看与组织者名单导出时对中间四位脱敏，检票端一律脱敏，本人查看自己的记录显示完整号码（REQ-029）。之所以在出口而不是在存储层脱敏，是因为线下收款与现场联系需要真实号码，存储一旦不可逆就没法支撑业务；权限判断放在导出与查询接口上。

日志表不提供 UPDATE 与 DELETE 的接口，账号层面只授予 INSERT 与 SELECT 权限；导出文件不落库保存内容，只保存元数据与文件路径，文件按快照生成时点的内容写死，避免下载时再查库导致人数与统计不一致（REQ-013、015）。

5 各功能模块的界面设计

界面按角色分两套载体：参与者端为移动端网页（画布 414×904），组织者、管理员与检票人员使用桌面工作台（画布 1240×760 与 1080×720）。九张图按亿图图示 / Figma / Pixso 这类工具的线框规范绘制（网格 8px、字号 10~14、控件命名与类图字段一致），本次用绘图脚本产出同规格 PNG，好处是尺寸与色板可整套复现，改稿时不会出现各张图不一致；尺寸统一、控件命名与类图字段一致，目的是让后面写代码时可以直接照着摆控件。九张图依次覆盖 M1 至 M10 十个模块中有界面的部分（M11 定时任务与 M12 公共契约没有界面，前者的执行结果在界面 5 与界面 8 上体现，后者是内部契约）：界面 1 对应 M1，界面 2 对应 M3，界面 3 与界面 4 对应 M4，界面 5 对应 M5 与 M9，界面 6 对应 M2 与 M8，界面 7 对应 M8 与 M10，界面 8 对应 M6，界面 9 对应 M7 与 M5 的对账入口。

5.1 参与者端



图 9　界面 1：登录、锁定提示与身份选择（M1）

功能要点：手机号与密码登录；注册时选择四类身份；组织者处于待审核状态时只能提交资质材料，不出现“创建活动”入口；账号锁定提示给出剩余时间与“联系管理员解锁”的下一步（REQ-001、002、025、028）。



图 10　界面 2：组合条件查询与结果列表（M3）

功能要点：七个条件（关键词、类别、主办社团、校区、日期区间、价格区间、状态）任意组合，结果按开始时间升序并显示剩余名额；日期区间或价格区间上下限颠倒时拒绝查询并保留已填条

件；空结果给出放宽条件的建议；状态条件里“已满”由剩余名额推导，管理员可勾选包含全部状态（REQ-006）。



图 11　界面 3：活动详情与场次、票种选择（M3、M4）



图 12　界面 4：确认提交与拒绝提示（M4、M12）

功能要点：报名固定三步，页首有步骤指示；详情页显示报名截止时间与服务器判定说明；票种列表同时给出剩余/总名额、单价与限购张数，已满票种直接给出候补入口；确认页展示本次提交使

用的一次性请求标识，字段校验失败时提示精确到字段；八类拒绝提示统一为“原因＋下一步”的格式，不出现无原因文案与堆栈信息（REQ-007、019、020、023、033）。



图 13　界面 5：我的报名与递补确认倒计时（M5、M9）

功能要点：递补待确认卡片带 30 分钟倒计时与进度条，超时后按钮失效并提示顺延结果；候补位次随队列变化更新；记录列表按状态给出可执行动作（待支付可取消、已签到不可取消）；本人记

录显示完整手机号，导出与检票端脱敏；取消操作有二次确认与原因说明（REQ-008、009、021、029）。

5.2 组织者与管理员工作台

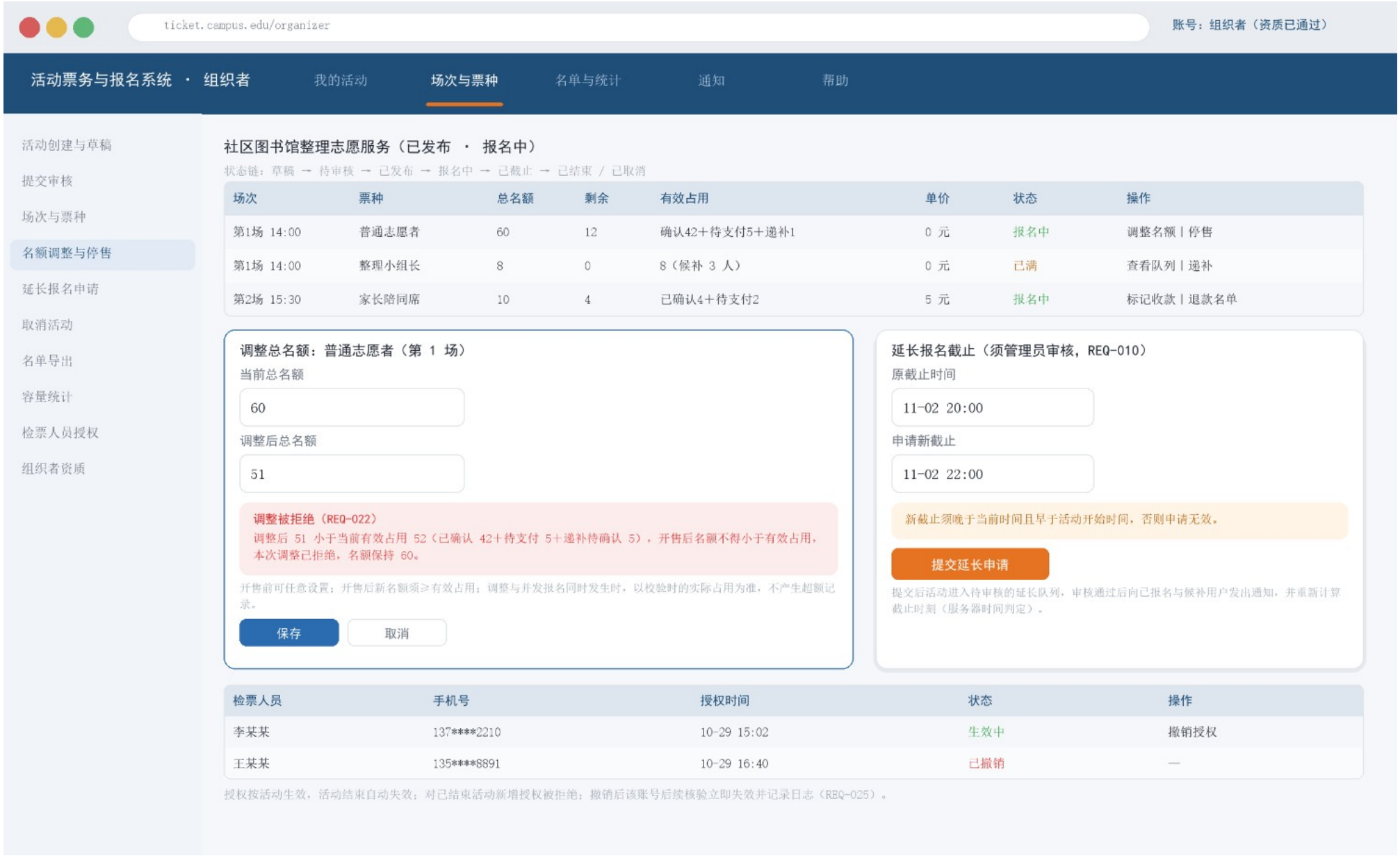


图 14　界面 6：组织者票种管理与名额调整下限校验（M2、M8）

功能要点：活动状态链按实验一的七态展示；票种表格显示总名额、剩余与有效占用的分项构成，调整名额被拒绝时提示当前有效人数与可行下限；延长报名截止时间只受理申请，审核通过后生效并向已报名与候补用户通知；检票人员授权表按活动生效、可撤销（REQ-005、010、022、025、026）。



图 15　界面 7：管理员审核队列与日志查询（M8、M10）

功能要点：四个审核队列以数量角标提示；关键信息变更用前后值对照表并标注是否关键项，审核期间详情页仍展示原信息；日志查询提供操作类型、操作人、活动、时间段四条件组合与导出，界面上没有修改与删除入口；撤销签到审批列明确写出审批后的状态（REQ-010、011、015、026）。

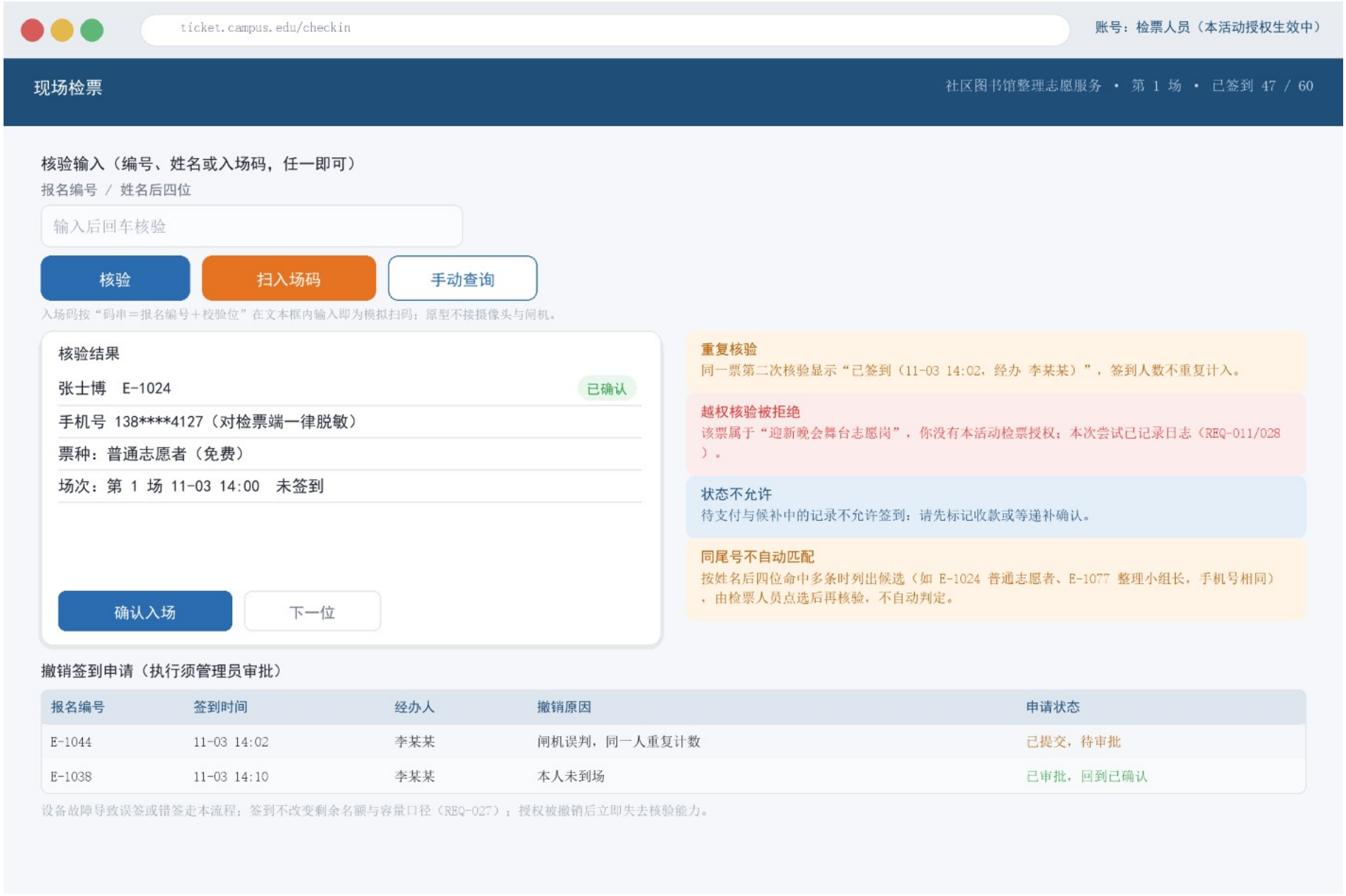


图 16　界面 8：检票端核验与撤销申请（M6）

功能要点：核验输入支持报名编号、姓名后四位与入场码，入场码在原型里以“编号＋校验位”的码串输入代替，不接摄像头与闸机；按姓名后四位命中多条时列出候选由检票人员点选，不自动取第一条；核验结果卡显示姓名、票种、场次与状态，手机号固定脱敏；重复核验给出已签到时间与经办人；越权核验拒绝并提示该票所属活动与授权范围；撤销签到只提供申请入口，不提供执行按钮（REQ-011、025、028、033）。

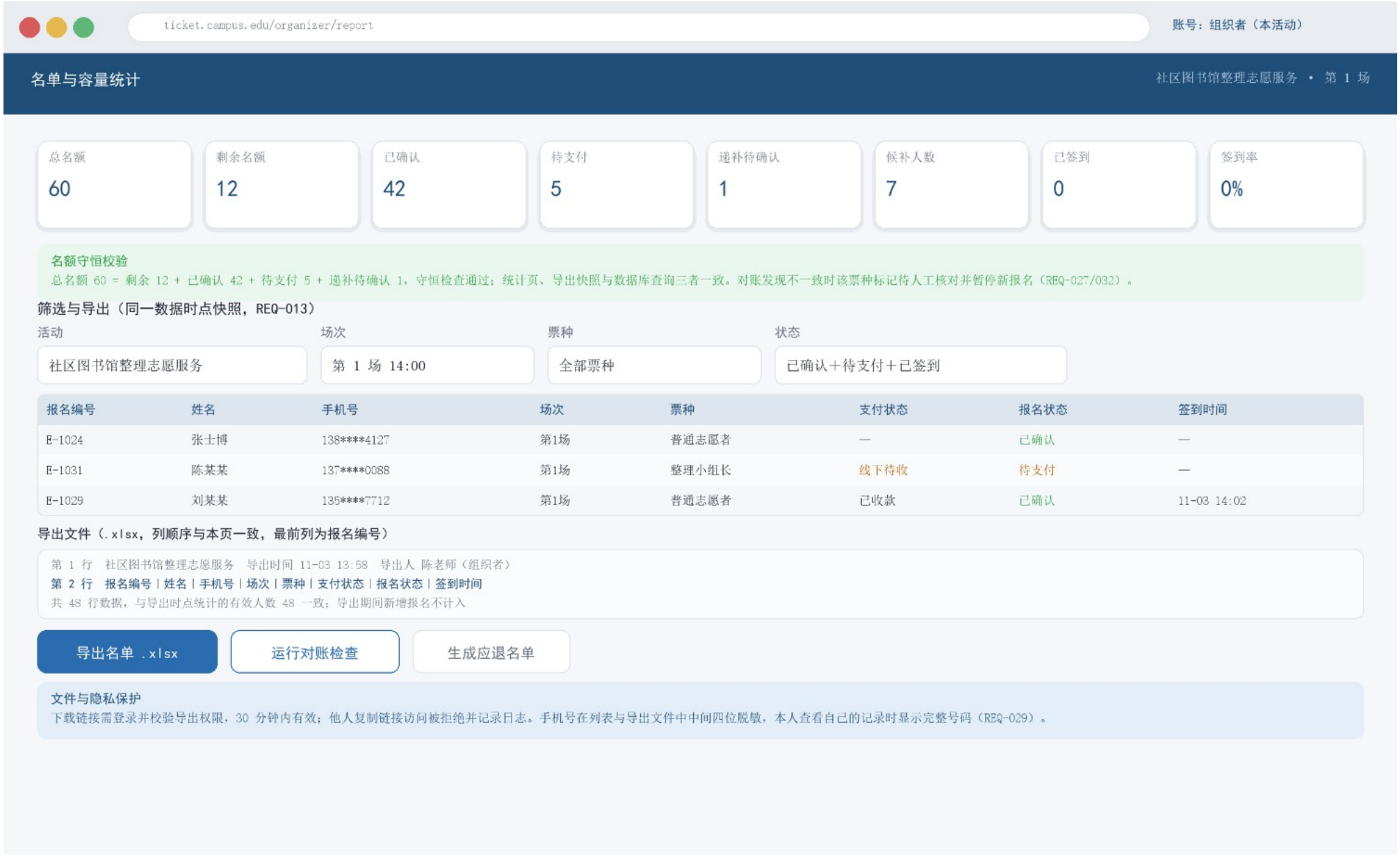


图 17　界面 9：容量统计与名单导出（M7、M5）

功能要点：八个统计数字分两行排列，守恒校验条用同一时点数据计算并给出等式；导出筛选按活动、场次、票种、状态四级；导出文件预览显示首行标注、列顺序与行数；对账按钮触发逐票种校验，发现不一致时提示该票种暂停新报名（REQ-013、027、029、032）。

九张界面的共同约定有三条：所有拒绝提示都带原因与下一步；所有时间显示标注“以服务器时间为准”；所有涉及手机号的显示遵循界面内脱敏、本人可见完整的规则。

5.3 界面元素的落地方式与实现取舍

任务书第五节要求设计尽可能保证基本功能的完备性（包括数据的存储形式），并明确学期结束时要交出一个 1500 行以上、基本结构完整、设计清晰的原型（任务书允许具体细节留作待办项）。界面稿不是画着好看的，每个控件都要能落到那样的原型上。下表逐项写出实现方式、依赖的接口或表，并标出原型阶段的取舍，避免到时候才发现某个控件做不出来。

表 8　界面元素与实现方式对照

界面元素或动作	原型实现方式	依赖接口 / 表	取舍与限制
登录、失败锁定与解锁	口令用 PBKDF2 加盐哈希存 password_hash；失败次数与锁定时间存 account，登录前先比较 lock_until	I_Auth、account	不做图形验证码；锁定提示的剩余秒数由服务端算好返回
七条件组合查询	服务端按非空条件拼 SQL，日期与价格区间先校验上下限再执行	查询接口、activity、ticket_type	关键词先用 LIKE 加前缀索引，一万活动量级够用，全文索引留给后续
三步报名与字段校验	前端做即时提示，提交时服务端逐条重算校验，前端校验不作数	I_Enroll、拒绝原因目录	字段只收需求内的姓名与手机号，不采集身份证号
一次性请求标识	进确认页时发标识，提交时带上，唯一索引冲突即返回第一次的结果	enrollment.request_token	不建独立令牌表，靠唯一约束实现幂等
页面版本号与“请刷新”	详情页返回 version，提交时带上，服务端比较后决定是否拒绝	activity.version、ticket_type.version	关键信息与名额规则变化时版本自增
名额不足转候补	同一事务内扣减失败则写候补队列并返回位次	I_Capacity、waitlist_position	截止后队列停止，候补入口给出原因而不是直接隐藏
30 分钟递补倒计时	服务端返回通知时刻与当前服务器时间，前端每秒用两者之差刷新；到期由定时任务顺延	I_Capacity、notice	前端只负责显示，判定全在服务器，改本机时间无效
剩余名额与守恒校验条	一次聚合查询出六项数字，页面按等式比对并显示校验结果	I_Capacity、ticket_type	统计与名单必须取同一时点，不允许各查一次
检票核验三种输入	报名编号、姓名后四位、入场码串（编号＋校验位）均在文本框输入	I_Checkin	原型不接摄像头与闸机，扫码用输入码串模拟
同尾号命中多条	查询返回候选列表，检票人员点选一条后再执行核验	I_Checkin	不自动取第一条，避免把别人的名额签掉
撤销签到	检票端只写申请，管理员审批通过后由服务端改状态并记日志	revoke_request、I_Checkin	撤销不改变名额，只回落状态
手机号脱敏显示	在查询与导出的出口处加工字符串，库里仍存明文	I_Export、查询接口	本人查看自己的记录走另一套口径，权限在服务端判
名单导出 xlsx	服务端按快照生成文件，返回带失效时间的下载地址	I_Export、export_snapshot、文件存储	文件落磁盘，库中只存时点、行数与失效时间
日志四条件查询	按操作类型、操作人、活动、时间段组合查询，页面只给查询与导出	I_Audit、audit_log	数据库账号只授 INSERT 与 SELECT，从权限层保证只追加
站内通知与未读角标	业务动作写 notice 表，页面定时拉未读数	通知服务、notice	外部短信渠道待确认，本期不做发送端
定时任务三项	原型用固定间隔轮询（每 30 秒）扫描到期递补、报名截止与缺席标记	M11	不引入消息队列，单机够用；每步都按服务器时间判定且可重入

6　需求与设计的对应核对

下表把 30 条纳入设计的需求逐条对到模块、类或表、界面与图上，用于自检是否漏项。作废的 REQ-014、REQ-016、REQ-017 不列入；待确认的 REQ-034、REQ-035 只列出预留位置。

表 9　需求与设计元素追踪矩阵

需求	模块	类 / 表 / 组件	界面	图
REQ-001	M1	Account、Organizer	界面 1	用例图、类图
REQ-002	M1	Account 角色子类、checkin_grant	界面 1、6、8	用例图、组件图
REQ-003	M2	Activity、activity 表	界面 6	类图、用例图
REQ-004	M8	ReviewService、qualification_review	界面 6、7	用例图、组件图
REQ-005	M2	ActivitySession、TicketType	界面 3、6	类图
REQ-006	M3	activity 索引、查询接口	界面 2	用例图、活动图
REQ-007	M4、M5	Enrollment、CapacityService	界面 3、4	活动图、时序图
REQ-008	M4、M5	Enrollment.requestCancel	界面 5	状态图、时序图
REQ-009	M5	WaitlistQueue、waitlist_position	界面 4、5	活动图、状态图
REQ-010	M8、M11	ExtensionRequest、ServerClock	界面 6、7	用例图、状态图
REQ-011	M6	CheckinService、revoke_request	界面 8、7	状态图、组件图
REQ-012	M2、M9	Activity.cancel、refund_list	界面 6	状态图、用例图
REQ-013	M7	ExportSnapshot、export_snapshot	界面 9	组件图、时序图
REQ-015	M10	AuditLog、audit_log	界面 7	包图、组件图、活动图
REQ-018	全层	查询索引、连接池、定时任务错峰	界面 2	包图（性能说明见 7.1）
REQ-019	M5	CapacityService.deductInTransaction	界面 4	时序图、活动图
REQ-020	M4	Enrollment.requestToken	界面 4	时序图、活动图
REQ-021	M5、M11	WaitlistPromotionJob	界面 5	时序图、状态图
REQ-022	M2	CapacityRule.checkLowerBound	界面 6	类图、界面说明
REQ-023	M4	Enrollment.pageVersion	界面 3、4	活动图、时序图
REQ-024	M4、M6	EnrollmentStateMachine	全部业务界面	状态图（含新增终止态）
REQ-025	M1、M8	CheckinGrant、qualification_review	界面 1、6、8	用例图、组件图
REQ-026	M8	KeyInfoChange	界面 6、7	用例图、组件图
REQ-027	M5	CapacityRule.invariantHolds	界面 9	类图注、时序图
REQ-028	M1	Account.verifyPassword、I_Sec	界面 1、8	包图、组件图
REQ-029	M7	脱敏出口、downloadExpireTime	界面 5、8、9	组件图、界面说明
REQ-030	M11	Enrollment.markAbsent	界面 7	状态图、活动图
REQ-031	M7	refund_list、paidAmount	界面 6、9	类图、状态图
REQ-032	M11	事务与对账任务	界面 9	时序图末段
REQ-033	M12	拒绝原因目录 C_Err	界面 4 及各处	包图、组件图
REQ-034	预留	不设计，仅留接口位	—	—（待甲方确认阈值）
REQ-035	M9	InsiteNotice、notice	界面 5、7	组件图（外部渠道预留）

7 大模型辅助设计的过程记录

本章记录大模型在这次设计里的使用情况。用之前先把自己要什么、什么算合格写清楚（比如“模块清单要覆盖 30 条有效需求、不得出现已作废条目”），拿到产出再逐条对着这个标准核，能满足要求才用，不满足就改或者推掉。用的范围是设计产物（模块候选清单、各类 UML 的 PlantUML 代码骨架、状态图漏项对照、界面控件清单、时序图并发写法）和报告的文字：正文的框架、口径和小标题我先定，模型按框架整理文字、提修改建议，采纳哪条由我核对后决定。下面五个场景分别记录使用场景、生成结果、优缺点分析和修改优化后的结果。

使用的工具与版本：DeepSeek 网页版，模型 DeepSeek-V3，会话模式，采样参数默认；绘图与渲染工具为 PlantUML 1.2026.10（本地 OpenJDK 17 渲染，非大模型）。交互时间为 2026 年 11 月 6 日 20:10 至 2026 年 11 月 10 日 22:40，共五次较集中的对话，逐条记录如下。

7.1 场景一：把需求清单转成模块候选

使用场景：开始设计前需要一个不漏的模块候选清单。输入给模型的是 requirements.md 里 35 条需求的编号、类型、状态三列（不带正文），要求按功能归并出模块并标注对应编号。

生成结果（摘录）：模型给出十一个模块，其中包含“在线支付模块（REQ-016）”“活动推荐模块（REQ-017）”，并把“候补队列”和“递补通知”合并成一个“消息中心”。

优缺点分析：优点是归并速度快、编号覆盖完整，把 REQ-018 的响应时间与 REQ-032 的故障恢复单独列成非功能模块，这一点我自己一开始没有想到；缺点很致命——REQ-016、REQ-017 在实验一已明确作废（不接线上支付、不做推荐），模型只看编号与状态列，没有读到“已拒绝”的实际含义，把作废需求当成了功能；把候补与通知合并也不成立，因为递补确认是要落库的业务状态，不是消息。

修改优化后的结果：删掉两条作废需求引出的模块，把“消息中心”拆回 M5 名额与候补、M9 通知两个模块，并按实验一那 30 条有效需求归类出的六类事情逐项核对，补上 M11 定时任务与 M12 公共契约，最终得到表 2 的十二个模块。核对方式是拿表 2 反向对 requirements.md 的编号列，看是否有编号无处安放。

7.2 场景二：类图的 PlantUML 代码骨架

使用场景：类图的字段和方法我已经在本子上列了一遍，想让模型把它转成 PlantUML 语法，节省手写代码的时间。输入是 Enrollment 与 TicketType 两个字段的中文草稿加 REQ-007、REQ-020、REQ-023 三条原文。

生成结果（摘录）：模型生成的类图里有 Enrollment 与独立的 WaitlistRecord 两个实体，候补信息放在 WaitlistRecord 的 status 上；Enrollment 没有 requestToken 与 pageVersion；票种上的名额字段命名为 capacity 与 left。

优缺点分析：优点是语法完全可用、关系箭头分得清楚（组合与关联没有混用），直接省掉我查 PlantUML 手册的时间；缺点有三处设计错误——第一，把候补拆成独立实体与 REQ-024 冲突，实验一明确候补中、递补待确认都是报名记录的状态，拆开后一条动作会出现两条记录，取消与递补

到底改哪条不清；第二，漏掉幂等标识与版本号，等于 REQ-020、REQ-023 没有落点；第三，capacity/left 的命名与需求文档的“总名额/剩余名额”对不上，而且没有体现 remaining_count 受 total_count 约束这条下限规则。

修改优化后的结果：删去 WaitlistRecord，把候补顺序抽成 waitlist_position 表与 WaitlistQueue 行为类，Enrollment 增补 requestToken、pageVersion、paidAmount 三个字段（第三个是 REQ-007 的单价快照要求，模型也没写，是我复查时补的）；字段名统一为 totalCount / remainingCount，并在 CapacityRule 里显式写出守恒与下限两个校验方法，见图 1 下方的注。

7.3 场景三：状态图的完整性检查

使用场景：画完状态图后心里没底，把 REQ-024 的原文（含转换清单与禁止项）整段发给模型，让它检查我的状态图有没有漏。

生成结果（摘录）：模型回答“八个状态与转换均完整，建议补充已退款到已取消的回退路径”。它给出的 PlantUML 片段照八个状态画，没有任何一处说明“候补中”在活动截止后的去向。

优缺点分析：这是五次对话里最典型的一次——模型做了字面校对，没做语义核对。它建议的“已退款→已取消”恰恰是 REQ-024 禁止的转换，如果照改就画错了；而真正的漏洞它没有发现：REQ-021 要求截止后剩余候补记录“转为已结束”，八个状态里没有这个终止态，候补记录在截止后无路可走。

修改优化后的结果：拒绝采纳它的回退建议，改为把 REQ-021 与 REQ-024 两段原文并排放在一起自己逐句对，确认需要第九个状态，命名为“候补已结束”，只接受候补中转入、不再转出、不占用名额，并在图内注明该状态来源于 REQ-021 与 REQ-024 清单不完全一致，建议在下一轮需求沟通时与甲方确认（按实验一约定应新增编号，不改动原条目），见 3.7 节。由此得出的经验是：让模型做“有没有漏”的判断不可靠，交叉引用两处原文的活儿只能自己做。

7.4 场景四：时序图的并发写法

使用场景：想验证抢名额的消息顺序写得对不对，把 REQ-019、REQ-020 原文与我的草图步骤一起发过去。

生成结果（摘录）：模型给的序列是 EnrollmentService 先调用 getRemaining 查询剩余名额，判断大于 0 之后调用 deduct 扣减，再 save 记录，异常时“补一次 cancel 回滚”。

优缺点分析：思路正确（分步骤清楚、角色命名规范），但这条序列本身是超卖的标准写法：查询与扣减之间有时间窗，两个并发请求都能查到剩余 1；把事务边界交给“失败时补一次回滚”也不是事务，日志写入失败与名额回滚会被拆成两件事，与 REQ-032 不符。模型的缺点在于它按“常见 Web 教程”的写法给方案，不考虑并发约束。

修改优化后的结果：改成对 ticket_type 行加锁后直接进入扣减与写入的同一事务，图上删除“先查后减”这一步；重复提交改为靠 request_token 唯一约束命中而不是查询判重；日志失败路径明确写成整个事务回滚且标识保持可用，与 30 分钟逾期顺延一起放在图内的注里（见图 8）。改完后再把新序列发回给模型复核一次，它这次只提了一个命名上的小问题，说明结构已站得住。

7.5 场景五：界面控件清单

使用场景：画界面前想让模型列一份参与者端与组织者端的控件清单，避免漏掉必要字段。

生成结果（摘录）：清单很全，列出了筛选栏、票种卡片、倒计时、名额表格等控件，并建议“名额不足时禁用报名按钮”。

优缺点分析：优点是控件命名与我的类图字段能对上，倒计时控件的提示是我原本漏想的（REQ-021 的时限要在界面上可见）；缺点有两条：禁用按钮的做法与 REQ-033 冲突——需求要求给出拒绝原因和下一步，按钮禁用等于什么也不说；另外它没有考虑手机号的显示口径，而 REQ-029 要求列表脱敏、本人可见完整号码、检票端一律脱敏，三处界面各不相同。

修改优化后的结果：名额不足时按钮保持可点，点击后给出“名额已满，可加入候补”的提示与候补入口（界面 4）；在我的报名页加 30 分钟倒计时与进度条（界面 5）；在界面 5、8、9 分别按三种口径处理手机号显示；组织者侧名额表格补上“有效占用”的分项列，使 REQ-022 的下限校验在界面上看得懂（界面 6）。

7.6 总结

五次辅助下来，模型在两类工作上确实省时间：一是把已经想清楚的东西转成规范语法（PlantUML 代码骨架、表清单排版），二是提供我没有预设的视角（把非功能需求单列为模块、倒计时控件）。在另外两类工作上必须警惕：一是它不读需求的状态含义，作废条目会被当功能捡回来；二是它给的方案偏向“通用写法”，与本项目的一致性、并发、时限类约束经常冲突，而这些约束恰恰是实验一花了两轮访谈才确认下来的，模型对此一无所知。我的处理办法是把模型的输出当成“待核对的草稿”，每次只问一个问题、只用一种产物，并且坚持拿两份需求原文交叉比对再自己复核——本设计里最重要的两处修改（候补实体的归并、第九状态的增设）都是这样发现的，模型在这两处一个没帮上忙、一个给了错误建议。归纳下来，这类工具适合放在设计链条的中段：把已经定下来的东西转成规范表达，把可能漏的地方摊开供人核对；不适合放在两头——前头是需求口径的判断，后头是结论对不对的签收。真正省时间的是拿自己写下的标准去检查它的产出，比自己从零想一遍整套设计快，也比照单全收安全。

7.7 能不能把流程设计成人只做检查校对

任务书希望我顺着这次体验想一件事：大模型能不能承担生成，人只负责检查校对。把五次对话按“人实际做了什么”拆开之后，我的结论是能做到大半，但不是全部，关键是把人能判的东西先写成机器也判得了的标准。下面这套是我这次实际用的流程，也是后面接着编码要用的。

（1）先写验收标准再发问。每次提问前用三行写清：要什么产物、给哪些输入、什么算合格。合格标准必须是可判定的句子，例如“模块清单要覆盖 30 条有效需求，编号为 REQ-014、016、017 的不得出现”“每个界面控件名要能在类图字段里找到”。

（2）让模型只交结构化产物，不外交代。要 PlantUML 源码、要建表语句、要控件清单表格，这三类东西能被机器读，它写“我先来分析一下”这类段落，一律删掉，否则检查成本反而变高。

（3）机器先过一遍，人不看第二遍就能判的绝不自己看。本次实际跑的三类检查：PlantUML 能否渲染通过、图里的类名与字段名是否出现在表 7 的字段里、界面稿里有无文字越界与重叠（脚本

记录每段文字的包围盒后比对）。这三项把 7.2 的命名不一致、第 5 章的排版问题全部在无人为干预下暴露出来。

（4）人只处理三类判断。语义是否与实验一口径冲突（如 7.1 的已作废需求、7.3 的禁止转换）；需求之间的空隙要不要补（如第九状态）；几个可行方案里选哪个（如候补位次落库还是动态算）。这三类无法写成判定句，只能人来签收。

（5）把人的结论写成“改哪一处”再交回模型，改完重跑第 3 步。本次 7.4 的时序图就是这么收敛的：第一轮给的是先查后减，我只回了“扣减与写入必须在同一事务、不得出现先查后减”，第二轮它只提了一个命名问题，说明结构已经站住。

按这套流程衡量，报告书写也是同一回事：小标题、要写的点、每点的结论由我先列出来，模型按列表成段，我检查的是编号是否对得上、口径是否与实验一一致、有没有把没做过的事写成做过的。这次报告里被我删掉的模型产出主要是两类：一是它把“建议”“可以考虑”写成结论，二是它会给没画过的图加解释。前者不能靠校对发现，只能靠事先把“只写已实现的，不写计划”写进标准；后者靠机器查引用（提到某张图就必须存在对应的图文件）就能拦住。

所以对人的工作量，我能减到“定标准、判口径、签收结论”三件事，但不能只剩最后一件。要真做到只检查校对，缺的是把这些判断也写成可执行的标准——这部分目前只有把需求文档读透的人能写，模型写不出来。

8 设计小结与后续

8.1 设计层面自认的不足

（1）交互性视图只有一张时序图，覆盖的是并发抢名额场景；名单导出与对账、撤销签到两条链没有单独画时序，组件图和 2.3 节的调用表只能说明“谁调谁”，说明不了时间上的交错。

（2）名额扣减依赖单库行锁，属于单机方案。REQ-018 的环境口径正是“4 核 8G 单机部署”，所以本设计不算跑题，但并发量真上去后需要改成按票种分片或预扣额度，这部分没有展开。

（3）REQ-035 的外部通知渠道未定，图上只画了 M9 的接口位置与降级到站内消息的路径，重试与送达判定的设计要等甲方确认后才补。

（4）界面为线框稿，没有做视觉规范（色板、字号阶、组件状态），若要接近成品还需补一轮视觉定义。

8.2 与实验一的衔接和对后续编码的交代

本设计没有改动实验一的任何需求编号与口径，只做了一处“按需求原文补齐结构”的处理，即 3.7 节说明的第九状态；除此之外，若在后续编码过程中发现设计需要调整（例如改为预扣额度、或候补位次改为动态计算不落库），属于正常现象，届时以编码实现的结论为准并在此报告中注明差异原因，不回头修改实验一已确认的需求条目。

课程组要求学期结束时交出一个 1500 行以上、基本结构完整、设计清晰的原型，后续实验的具体要求还没公布；就“设计要能直接拿去写代码”这一点，本设计留了三处落点：表 7 的字段可以

照搬成建表语句；3.2 节的六个接口即原型需要实现的方法名；表 9 的追踪矩阵可以在实现完成后逐行回填“已实现/未实现”，用来检查是否漏做需求。

8.3 提交物清单

表 10　压缩包内的文件与格式

文件	格式	内容
实验报告.pdf	PDF	本报告正文，含八张 UML 图与九张界面图
UML*.puml	PlantUML 源码	八张图的源码，UTF-8 无 BOM、LF 换行，可本地重新渲染
UML*.png	PNG	与源码同名的八张渲染图
UI 界面\界面 1~9*.png	PNG	九张界面线框稿（参与者端 414×904，工作台 1240×760 / 1080×720，2 倍尺寸导出）

八张图均可用命令 java -jar plantuml.jar -tpng -charset UTF-8 UML*.puml 重新生成，渲染不依赖在线服务；每张 .puml 的头部注释标出了它覆盖的需求编号，便于回查。

修改过程按课程要求用 git 分次记录：一次提交放模型生成的初稿（.puml 源码与按控件清单排出的界面稿），一次提交放人工核对后的改动，包括 7.1 删掉的作废需求、7.2 合并的候补实体、7.3 补的第九状态、7.4 改的事务顺序，以及第 5 章被脚本查出并修掉的排版问题；压缩包内是最后一次人工核对后的版本。